
LogicWard — Demo Guide

How to run the demo, and read every detection

A step-by-step presenter runbook: which script to run from where, what the attacker does, where it lands, how LogicWard detects it, which dashboard view shows it, and the exact MITRE ATT&CK for ICS technique behind each alert.

The whole demo is one command:

```
python -m logicward.attacker.demo_sequence
```

Open **<http://localhost:8080/>** (login soc/soc123) and watch the attacks land. A full run produces **14 events** — **4 critical, 2 high, 3 medium, 5 low**.

Contents

1	The cast — who is who, and where things run	2
2	Running the demo	2
2.1	Option A — one command (recommended for judging)	2
2.2	Option B — drive it yourself	2
2.3	Option C — the real two-machine setup	2
3	The story, act by act	3
3.1	Act 1 — Normal operations	3
3.2	Act 2 — Reconnaissance: a rogue device	3
3.3	Act 3 — Silent logic tampering	3
3.4	Act 4 — Process manipulation & impact	4
3.5	Act 5 — Physical tamper	5
3.6	Detection summary	5
4	Where each detection shows on the dashboard	5
5	Judge Q&A	5
6	Master map — attack → detection → MITRE	7

1 The cast — who is who, and where things run

LogicWard has three roles. In the **one-command demo** all three run on your laptop (an *embedded* PLC); in the **real deployment** they are three separate machines on the same Wi-Fi. The detection logic is identical either way.

Role	What it is	Where it runs
The PLC (victim)	Thermal power-plant controller: a Modbus TCP server + its ladder program (a real Rockwell .L5X) + program-download endpoint.	Raspberry Pi (real) <i>or</i> embedded in the demo process (laptop).
The Attacker	A toolkit that tampers with the PLC: setpoint writes, program downloads, a Modbus flood, a rogue device.	<code>logicward.attacker</code> — a 2nd box (real) <i>or</i> the demo script (laptop).
LogicWard (defender)	The detection engine + SOC dashboard that diffs the live PLC against a signed baseline and raises alerts.	The laptop — <code>localhost:8080</code> .

The mental model for the audience: “An attacker who can reach a power-plant PLC quietly rewrites its safety logic. LogicWard sits beside the PLC, holds a cryptographically-signed copy of the approved program, and the instant the running logic or a live value drifts from that baseline it raises a severity-ranked, MITRE-mapped alert — naming what changed, and how.”

2 Running the demo

2.1 Option A — one command (recommended for judging)

From the project root on the laptop:

```
python -m logicward.attacker.demo_sequence          # ~90s, watchable pacing
python -m logicward.attacker.demo_sequence --fast   # ~15s, quick run
```

This brings up the SOC dashboard + an embedded thermal PLC + the program endpoints, then runs the whole attack story with narration in the terminal. Open **`http://localhost:8080/`** (login `soc/soc123`) and keep it on screen.

2.2 Option B — drive it yourself

```
# Terminal 1 - the SOC dashboard (embedded PLC)
python -m logicward.dashboard.app

# Terminal 2 - the program endpoints (so program-download attacks land)
python -m logicward.plant.logic_store

# Terminal 3 - fire attacks one at a time (see the catalogue in section 4)
python -m logicward.attacker.attacks logic-inversion
python -m logicward.attacker.attacks ddos --count 800
```

2.3 Option C — the real two-machine setup

Pi runs the PLC + sensor agent; the laptop runs the dashboard; a third box attacks. Full runbook in **USAGE.md §7**. In short:

```
# On the Pi:      bash deploy/pi_bootstrap.sh <laptop-ip>  &&  bash deploy/run_pi.sh
```

```
# On the laptop: .\deploy\run_laptop.ps1 -PiHost siddhesh-pi.local
# From attacker: python -m logicward.attacker.attacks --host siddhesh-pi.local logic-
inversion
```

Say: “Everything you’re about to see is two computers talking over Wi-Fi — an attacker on one, the plant and its guardian on the other. For this screen it’s one laptop, but the code path is identical.”

3 The story, act by act

Each act below lists: the **command**, where the attack lands, how **LogicWard** detects it, which **dashboard view** shows it, the **MITRE ATT&CK** for ICS technique, and a **line to say**.

3.1 Act 1 — Normal operations

Nothing is attacked. The plant runs nominal, the baseline is HMAC-locked, the live program matches the baseline, the alert feed is empty.

- **Dashboard:** *Overview* — baseline integrity **VALID**, “program in sync”, zero alerts. *Live Plant* shows drum level, steam pressure, turbine speed all nominal.

Say: “A thermal power plant PLC, continuously verified against a cryptographically-signed approved program. Quiet board, green baseline.”

3.2 Act 2 — Reconnaissance: a rogue device

Attack: an unapproved engineering laptop joins the OT network.

- **Command:** `python -m logicward.attacker.attacks rogue`
- **Lands on:** the OT LAN segment (an ARP announcement) — not the PLC yet.
- **Detected by:** `arp_watch` (physical plane, on the Pi agent) — MAC not on the approved allowlist.
- **Event:** `physical.rogue_device` medium
- **Shown on:** *Alerts* feed.
- **MITRE:** Initial Access • **Rogue Master** • T0848

Say: “Before touching the PLC, the attacker’s machine appears on the OT segment. An unknown MAC on this network is itself an alert — Rogue Master, T0848.”

3.3 Act 3 — Silent logic tampering

The core of the demo: five ways an attacker rewrites the PLC’s safety logic.

(3a) Setpoint drift — over Modbus.

- **Command:** `python -m logicward.attacker.attacks setpoint-drift` (drum-level trip 220 -> 40 mm)
- **Lands on:** a Modbus holding register on the PLC (function code 06 write).
- **Detected by:** the drift engine’s **register diff** (live Modbus vs baseline snapshot).

- **Event:** `cyber.setpoint_drift` **high** (safety-critical setpoint)
- **Shown on:** *Alerts*, and the changed value on *Program Diff* / *Live Plant*.
- **MITRE:** Impair Process Control • **Modify Parameter** • T0836

(3b–3d) Program downloads — structural logic changes. Each command pushes a tampered .L5X to the PLC’s program-download endpoint; LogicWard canonicalizes and diffs it, and the passive FIM also fires on the file change.

- **Logic inversion** — attacks `logic-inversion` — flips the drum-level trip compare LES→GRT (the interlock now fires backwards).
Event `cyber.logic_inversion` **critical** • Persistence • **Modify Program** • T0889
- **Condition stripping** — attacks `condition-stripping` — removes the running-interlock from the furnace-flame trip (a safety input deleted).
Event `cyber.condition_stripping` **critical** • Persistence • **Modify Program** • T0889
- **Coil hijack** — attacks `coil-hijack` — repoints the feedwater-trip output coil to a different actuator.
Event `cyber.coil_hijack` **critical** • Persistence • **Modify Program** • T0889

(3e) Rung injection — a backdoor rung.

- **Command:** `python -m logicward.attacker.attacks rung-injection` (unlatches the turbine trip)
- **Detected by:** structural L5X diff (a new, unapproved rung appears) + FIM.
- **Event:** `cyber.rung_injection` **critical**
- **Shown on:** *Program Diff* — the new rung highlighted green.
- **MITRE:** Lateral Movement • **Program Download** • T0843

Each program download also raises `cyber.program_file_modified` **low** from the **FIM** sensor (Persistence • Modify Program • T0889) — a second, independent detection of the same tamper via the file system.

Say (on Program Diff): “This is the exact logic flip — the drum-low trip compare inverted from LES to GRT. A harmless re-export with new timestamps would show *nothing* here; a single changed instruction lights up red. That’s canonicalization doing its job.”

3.4 Act 4 — Process manipulation & impact

(4a) Force a control coil.

- **Command:** `python -m logicward.attacker.attacks force-coil` (forces the fuel-valve coil over Modbus)
- **Detected by:** register diff (a control coil forced away from baseline).
- **Event:** `cyber.register_change` **low**
- **MITRE:** Impair Process Control • **Unauthorized Command Message** • T0855

(4b) DDoS flood.

- **Command:** `python -m logicward.attacker.attacks ddos -count 800`
- **Lands on:** the PLC’s Modbus server (request flood).

- **Detected by:** the **resource plane** (CPU spike on the PLC host).
- **Event:** `resource.cpu_spike` **medium**
- **MITRE:** Inhibit Response Function • **Denial of Service** • T0814

3.5 Act 5 — Physical tamper

Attack: the Ethernet cable is pulled and the enclosure door is opened.

- **Detected by:** `link_watch` (carrier lost) and `gpio_tamper` (door switch) — physical plane, on the Pi agent.
- **Events:** `physical.link_down` **medium** • `physical.enclosure_open` **high**
- **MITRE:** `link_down` → Denial of Service T0814; enclosure tamper has *no* direct ICS technique (shown honestly as N/A).

Say: “The same fabric that caught a logic edit also catches a pulled cable and an opened cabinet — one event contract across cyber, physical, and resource.”

3.6 Detection summary

After a full run the dashboard shows **14 events**: **critical** ×4 **high** ×2 **medium** ×3 **low** ×5.

Every event is in the append-only evidence log — export the **signed PDF forensic report** from the *Evidence* view. **Say:** “One click — a signed incident report an investigator can trust.”

4 Where each detection shows on the dashboard

	View	Wh
	Overview	Base strip
	Program Diff	The wor
Login soc/soc123. Roles: operator (read-only), engineer (+baseline), soc (everything).	Alerts	Use The and
	Live Plant	The turb
	Evidence	The

5 Judge Q&A

- **“Is the PLC program real?”** Yes — a genuine Rockwell .L5X export. We never *execute* it; we parse, canonicalize (strip volatile dates/revisions/CRC/comments), and diff it. That’s why a benign re-export never false-alarms but a single inverted instruction is caught.
- **“How do you know the baseline itself wasn’t tampered?”** It’s HMAC-SHA256 signed; editing the approved copy breaks the signature and raises a **critical** `baseline_tamper` on its own — and a passive file-integrity monitor watches it in real time.

- **“Real MITRE or made up?”** Rule-based, explainable, and *verified* against the live ATT&CK for ICS matrix. Where no technique fits (physical enclosure tamper) we say N/A rather than invent an ID.
- **“Does it need the Pi?”** No — it runs entirely on one laptop with an embedded plant for judging, and splits cleanly onto a real Pi + attacker box over Wi-Fi with the same code.
- **“How is it verified?”** 98 automated checks across 7 suites (`python -m logicward.tests.smoke_*`).

6 Master map — attack → detection → MITRE

Attack	Command (attacker)	Lands on / channel	Detected as (plane)	Dashboard view	MITRE ATT&CK for ICS	Sev.
Rogue device	<code>attacks rogue</code>	OT LAN / ARP announce	<code>physical.rogue_device</code> (physical)	Alerts	Initial Access • Rogue Master • T0848	medium
Setpoint drift	<code>attacks setpoint-drift</code>	PLC Modbus HR / FC06 write	<code>cyber.setpoint_drift</code> (cyber)	Alerts, Program Diff, Live Plant	Impair Process Control • Modify Parameter • T0836	high
Logic inversion	<code>attacks logic-inversion</code>	PLC program download	<code>cyber.logic_inversion</code> (cyber)	Program Diff, Alerts	Persistence • Modify Program • T0889	critical
Condition stripping	<code>attacks condition-stripping</code>	PLC program download	<code>cyber.condition_stripping</code> (cyber)	Program Diff, Alerts	Persistence • Modify Program • T0889	critical
Coil hijack	<code>attacks coil-hijack</code>	PLC program download	<code>cyber.coil_hijack</code> (cyber)	Program Diff, Alerts	Persistence • Modify Program • T0889	critical
Rung injection	<code>attacks rung-injection</code>	PLC program download	<code>cyber.rung_injection</code> (cyber)	Program Diff, Alerts	Lateral Movement • Program Download • T0843	critical
(any program download)	— (side effect of 3b-3e)	PLC program file	<code>cyber.program_file_modified</code> (FIM)	Alerts	Persistence • Modify Program • T0889	low
Force coil	<code>attacks force-coil</code>	PLC Modbus coil / FC05 write	<code>cyber.register_change</code> (cyber)	Alerts, Live Plant	Impair Process Control • Unauthorized Command Message • T0855	low
DDoS flood	<code>attacks ddos -count 800</code>	PLC Modbus server / flood	<code>resource.cpu_spike</code> (resource)	Alerts	Inhibit Response Function • Denial of Service • T0814	medium
Cable pull	(physical / <code>link_watch</code>)	PLC NIC carrier	<code>physical.link_down</code> (physical)	Alerts	Inhibit Response Function • Denial of Service • T0814	medium
Enclosure open	(physical / <code>gpio_tamper</code>)	PLC cabinet switch	<code>physical.enclosure_open</code> (physical)	Alerts	Initial Access • (no direct technique) • N/A	high

Severity is computed per event type (base weight × a safety-critical multiplier), not hand-set. A full `demo_sequence` run = 14 events (4 critical, 2 high, 3 medium, 5 low).